Algoritmos y su implementación

P2A8 | Dylan Herrera | Universidad Anáhuac Mayab
11 de noviembre de 2026

Para esta actividad elegí dos situaciones: buscar libros en una biblioteca antes de que existieran los catálogos digitales y buscar productos en una tienda en línea. Aunque son de épocas distintas, en las dos hace falta organizar los datos para encontrar lo que se necesita. Voy a relacionarlas con inserción, merge sort y tablas hash.

Caso 1 Buscar libros en una biblioteca

¿Cuál es el problema?

El problema es encontrar un libro entre muchos y saber en qué estante está. Para hacerlo, la biblioteca necesita registrar el título, el autor, el tema y la ubicación. Si estos datos no están organizados, buscar un solo libro puede tomar bastante tiempo.

¿En qué época surgió o desde cuándo existe?

Este problema ya existía en el siglo XIX. Por ejemplo, en la Biblioteca del Congreso de Estados Unidos aumentó mucho la llegada de materiales después de 1870, cuando asumió funciones relacionadas con el depósito de derechos de autor. Los catálogos impresos se quedaron atrás y, a principios de 1900, se abrió un nuevo catálogo de fichas al público. [1]

¿Quiénes enfrentaban esta situación?

Lo enfrentaban los bibliotecarios, que tenían que registrar y acomodar la información, y los lectores e investigadores que buscaban las obras. Mientras más crecía la colección, más difícil era mantener todo actualizado.

¿Qué dificultades presentaba el manejo de información?

Cada libro nuevo podía dejar incompleto el catálogo impreso. También había problemas si un nombre o un título se escribía de distintas formas. Las fichas ocupaban espacio y tenían que colocarse en el lugar correcto; una ficha mal acomodada podía complicar la búsqueda.

¿Cómo se resolvía originalmente?

Primero se utilizaban catálogos impresos y suplementos. Después, el sistema de fichas permitió agregar registros sin imprimir todo otra vez. La Biblioteca del Congreso usó entradas por autor y tema, y en 1901 empezó a distribuir fichas a otras bibliotecas para que no repitieran el mismo trabajo. [1, 2]

Algoritmos para el catálogo

Algoritmo básico ordenamiento por inserción

Usaría inserción cuando llegan pocas fichas nuevas y el resto ya está ordenado. La idea es comparar cada ficha con las existentes hasta encontrar dónde colocarla. Por ejemplo, si están Algorithms y Pedro Páramo, Dune va entre los dos. Ordenar un arreglo completo cuesta $O(n^2)$ en promedio y en el peor caso; si ya está ordenado, cuesta $O(n)$. Agregar un elemento puede necesitar hasta $O(n)$ comparaciones o desplazamientos. [6]

Algoritmo avanzado ordenamiento por mezcla

Si hubiera muchas fichas desordenadas, elegiría merge sort. Se divide el conjunto en partes, se ordenan y después se van uniendo. Su tiempo es $O(n \log n)$, por lo que conviene más para cantidades grandes. Además, una implementación estable mantiene el orden previo de los registros que tienen la misma clave. La versión con arreglos necesita $O(n)$ de espacio extra. Aplicarlo a fichas físicas es mi propuesta para entender el proceso, no un método histórico confirmado. [6]

¿Cómo se utilizaría una tabla hash?

Si se digitalizan los registros, cada ejemplar puede tener un identificador único. Ese identificador sería la clave de la tabla hash, y el valor guardaría el título, autor y ubicación. La función hash indica en qué cubeta buscar. Si dos claves caen en la misma, se guardan en una lista y se comparan completas. Para buscar por título habría que guardar varios resultados, porque puede haber distintas ediciones o ejemplares del mismo libro.

Alternativa	Ventajas	Desventajas
Inserción	Sencilla; útil para grupos pequeños o casi ordenados; estable.	Tiempo cuadrático para muchos registros desordenados; desplaza elementos en arreglos.
Mezcla	Tiempo $O(n \log n)$ garantizado; estable; permite combinar grupos.	Necesita espacio auxiliar; reordenar todo resulta innecesario para una sola ficha.
Tabla hash	Búsqueda exacta esperada $O(1)$ con buena distribución y carga controlada.	No ordena alfabéticamente ni busca por intervalos; requiere memoria y manejo de colisiones. Peor caso $O(n)$.

¿Cuál sería la mejor opción y por qué?

Yo usaría inserción para agregar pocas fichas a un catálogo físico que ya está ordenado, y merge sort para ordenar una colección grande desde cero. En un catálogo digital combinaría el ordenamiento con una tabla hash para consultar el identificador. Así se encuentra rápido el registro y luego se revisa la ubicación del libro.

Caso 2 Buscar productos en una tienda en línea

¿Cuál es el problema?

Una tienda en línea tiene que encontrar productos por su código SKU y mostrarlos ordenados, por ejemplo, de menor a mayor precio. Si revisa todo el catálogo cada vez que alguien busca algo, el trabajo aumenta conforme se agregan productos. Para este caso usaré una tienda como ejemplo y PostgreSQL como evidencia de un sistema real que utiliza estas técnicas.

¿En qué época surgió o desde cuándo existe?

Esta versión del problema se relaciona con las bases de datos y el comercio por internet de finales del siglo XX. Como referencia, POSTGRES empezó a implementarse en 1986, incorporó SQL en 1994 y adoptó el nombre PostgreSQL en 1996. Son fechas de la evolución de ese sistema; el control de inventarios existía desde antes. [3]

¿Quiénes enfrentan esta situación?

Lo enfrentan los clientes al buscar productos, el personal que revisa las existencias y los programadores y administradores que mantienen la base de datos. También puede haber varias personas consultando o modificando la información al mismo tiempo.

¿Qué dificultades presenta el manejo de información?

Hay que manejar muchos registros sin tardar demasiado. Los precios y las existencias cambian, puede haber nombres repetidos y algunos resultados no caben en la memoria disponible para ordenarlos. Además, buscar un SKU exacto no es lo mismo que pedir todos los productos entre dos precios.

¿Cómo se resolvía originalmente?

Antes de los sistemas digitales, el inventario se llevaba con registros en papel y revisiones físicas. Una solución informática sencilla sería guardar los productos en una tabla y recorrerla hasta encontrar el código. Esa búsqueda secuencial cuesta O(n). A partir de ese caso básico se entiende por qué conviene agregar índices cuando hay muchos datos.

Evidencia de aplicación en un sistema real

En la documentación de PostgreSQL se explica que los índices hash sirven para comparar valores por igualdad. Su código también describe cómo ordena conjuntos grandes mediante archivos temporales y una mezcla de varias secuencias ordenadas. Esto muestra una aplicación real de hashing y mezcla, aunque no significa que todas las tiendas usen exactamente la misma solución. [4, 5]

Algoritmos para los productos

Algoritmo básico ordenamiento por inserción

Usaría inserción para una lista pequeña de productos. Por ejemplo, con precios de 900, 250 y 600, primero se mueve 250 antes de 900 y luego se coloca 600 entre ambos. El SKU puede servir para desempatar precios iguales. Es fácil de implementar, pero su costo promedio y peor caso de $O(n^2)$ no conviene para millones de registros. [6]

Algoritmo avanzado ordenamiento externo por mezcla

Para ordenar un reporte grande que no cabe en memoria, elegiría mezcla externa. Se ordenan grupos que sí caben, se guardan temporalmente en disco y después se mezclan. Por ejemplo, las secuencias [250, 900] y [400, 600] se unen en [250, 400, 600, 900]. La ventaja es que no se necesita cargar todo a la vez; la desventaja es el espacio temporal y el acceso a disco. El código incluido muestra merge sort en memoria, no la versión externa.

¿Cómo se utilizaría una tabla hash?

Usaría el SKU como clave y el registro del producto como valor. La tabla se construye una vez y se actualiza cuando cambian los datos. Con una buena distribución de claves y una carga controlada, buscar cuesta O(1) esperado y construir la tabla O(n) esperado. Si hay colisiones, se compara el SKU completo. Esta estructura sirve para búsquedas exactas, pero no devuelve los productos ordenados ni busca por rangos de precio.

Alternativa	Ventajas	Desventajas
Inserción	Poco costo inicial en grupos pequeños; estable; O(1) de espacio de trabajo si se modifica el arreglo existente.	Tiempo promedio y peor caso $O(n^2)$; no conviene para un catálogo grande desordenado.
Mezcla externa	Trabaja con memoria limitada; procesa secuencias; útil para resultados ordenados grandes.	Requiere disco temporal, entrada y salida y posibles pasadas adicionales.
Hashing	Búsqueda exacta esperada rápida por SKU; evita ordenar para cada consulta puntual.	No permite ordenar ni consultar intervalos; requiere actualizar el índice y resolver colisiones.

¿Cuál sería la mejor opción y por qué?

Para este caso combinaría una tabla hash para buscar por SKU y mezcla externa para generar un listado grande ordenado. También consideraría un B-tree en una base de datos real: permite rangos y puede garantizar que no se repita el SKU, algo que el índice hash de PostgreSQL no hace. Si ya existe un índice con el orden adecuado, podría evitarse ordenar de nuevo. [4, 7]

Ejemplo en Python

Incluí un ejemplo en Python 3 con libros y productos. Para ejecutarlo, se guarda el código como algorithms_demo.py y se usa `python3 algorithms_demo.py`. Los datos son inventados para mostrar cómo funcionan los algoritmos.

¿Qué demuestra el código?

El programa tiene inserción, merge sort estable y una tabla hash con listas para resolver colisiones. Ordena los libros por título y los productos por precio y SKU. Ambas funciones crean una lista nueva, así que la versión de inserción del ejemplo usa O(n) de espacio para la copia, aunque ordenar directamente sobre el arreglo solo necesita O(1) de espacio de trabajo.

Para que la tabla hash sea fácil de seguir, la función suma los bytes de la clave y obtiene el residuo entre el número de cubetas. Esta función es básica y puede causar muchas colisiones. La tabla tampoco cambia de tamaño. En una aplicación real habría que mejorar la distribución y redimensionarla cuando aumente la carga.

Pruebas de corrección

Las pruebas comparan los dos ordenamientos con sorted de Python. También revisan listas vacías, un solo dato, datos invertidos, repetidos y ya ordenados. Para la tabla hash se prueban claves inexistentes, colisiones y cambios sobre una clave existente. Todas las comprobaciones pasaron; no se midieron tiempos de ejecución.

Salida observada del programa

```
Libros ['B01', 'B02', 'B03']
Consulta B02 {'id': 'B02', 'title': 'Dune'}
Productos ['P01', 'P02', 'P03']
Consulta P02 {'id': 'P02', 'price': 600}
Todas las pruebas de corrección pasaron.
```

Escala y limitaciones

Con un millón de registros, n^2 es 1 000 000 000 000, mientras que n log2(n) es aproximadamente 19 931 569. Estos valores sirven para comparar el crecimiento de los algoritmos, no son tiempos ni conteos exactos. Aun así, ayudan a ver por qué un ordenamiento cuadrático se vuelve poco práctico cuando aumentan los datos.

Conclusión

Lo que cambia entre los dos casos es la cantidad de información y la forma de guardarla, pero en ambos hace falta poder encontrar los registros. Usaría inserción para pocos elementos, merge sort para ordenar grupos grandes y hashing para búsquedas exactas. Para mí, la mejor solución es combinar los métodos según lo que se necesita hacer, en vez de usar uno para todo.

Fuentes consultadas

Consultadas el 11 de noviembre de 2026. Los números del texto corresponden a esta lista. Títulos traducidos al español.

[1] Biblioteca del Congreso de Estados Unidos. *Orígenes y desarrollo del catálogo de fichas.*
https://guides.loc.gov/card-catalog/card-catalog-history
Historia de los catálogos impresos y transición al catálogo público de fichas.

[2] Biblioteca del Congreso de Estados Unidos. *Anuncio del Servicio de Distribución de Catalogación del 4 de octubre de 1993.*
https://www.loc.gov/loc/lcib/93/9318/cds.html
Documenta la distribución de fichas desde 1901 y el uso posterior de registros MARC.

[3] Grupo de Desarrollo Global de PostgreSQL. *Breve historia de PostgreSQL.*
https://www.postgresql.org/docs/current/history.html
Fechas del proyecto POSTGRES, la incorporación de SQL y el nombre PostgreSQL.

[4] Grupo de Desarrollo Global de PostgreSQL. *Índices hash.*
https://www.postgresql.org/docs/current/hash-index.html
Consultas de igualdad, desbordamiento de cubetas y ausencia de verificación de unicidad.

[5] Colaboradores de PostgreSQL. *Código fuente de tuplesort c.*
https://doxygen.postgresql.org/tuplesort_8c_source.html
Los comentarios iniciales explican los archivos temporales y la mezcla equilibrada de múltiples secuencias. El código de desarrollo puede cambiar.

[6] Robert Sedgewick y Kevin Wayne Universidad de Princeton. *Ordenamiento y búsqueda.*
https://introcs.cs.princeton.edu/java/42sort/
Explicaciones del ordenamiento por inserción y mezcla y su crecimiento computacional.

[7] Grupo de Desarrollo Global de PostgreSQL. *Tipos de índices.*
https://www.postgresql.org/docs/current/indexes-types.html
Distingue el acceso por igualdad y rangos de B-tree del acceso por igualdad de hash.

Código en Python

```
# Python 3. No requiere paquetes externos.
def insertion_sort(items, key):
    a = list(items)
    for i in range(1, len(a)):
        current = a[i]
        j = i - 1
        while j >= 0 and key(a[j]) > key(current):
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = current
    return a


def merge_sort(items, key):
    if len(items) <= 1:
        return list(items)
    mid = len(items) // 2
    left = merge_sort(items[:mid], key)
    right = merge_sort(items[mid:], key)
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if key(left[i]) <= key(right[j]):
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    return result + left[i:] + right[j:]


class HashTable:
    # Encadenamiento: comparar claves completas en cada cubeta.
    def __init__(self, size=7):
        self.buckets = [[] for _ in range(size)]

    def _index(self, key):
        return sum(str(key).encode("utf-8")) % len(self.buckets)

    def put(self, key, value):
        bucket = self.buckets[self._index(key)]
        for i, (stored, _) in enumerate(bucket):
            if stored == key:
                bucket[i] = (key, value)
                return
        bucket.append((key, value))

    def get(self, key):
        for stored, value in self.buckets[self._index(key)]:
            if stored == key:
                return value
        return None
```

Código en Python continuación

```
def demo():
    books = [
        {"id": "B03", "title": "Pedro Paramo"},
        {"id": "B01", "title": "Algorithms"},
        {"id": "B02", "title": "Dune"},
    ]
    products = [
        {"id": "P03", "price": 900},
        {"id": "P01", "price": 250},
        {"id": "P02", "price": 600},
    ]
    for label, rows, key, lookup_id in [
        ("Libros", books, lambda r: r["title"].casefold(), "B02"),
        ("Productos", products, lambda r: (r["price"], r["id"]), "P02"),
    ]:
        basic = insertion_sort(rows, key)
        advanced = merge_sort(rows, key)
        assert basic == advanced == sorted(rows, key=key)
        table = HashTable()
        for row in rows:
            table.put(row["id"], row)
        print(label, [row["id"] for row in advanced])
        print("Consulta", lookup_id, table.get(lookup_id))
        assert table.get("missing") is None
    # Probar entradas vacías, claves iguales, colisiones y cambios.
    for values in [[], [1], [3, 2, 1], [2, 1, 2], [1, 2, 3]]:
        assert insertion_sort(values, lambda x: x) == sorted(values)
        assert merge_sort(values, lambda x: x) == sorted(values)
    equal = [(2, "first"), (1, "other"), (2, "second")]
    for sorter in (insertion_sort, merge_sort):
        assert sorter(equal, lambda x: x[0]) == [equal[1], equal[0], equal[2]]
    table = HashTable(1)  # Forzar todas las claves a una sola cubeta.
    table.put("A", 10); table.put("B", 20); table.put("A", 30)
    assert table.get("A") == 30 and table.get("B") == 20
    assert table.get("C") is None
    print("Todas las pruebas de corrección pasaron.")


if __name__ == "__main__":
    demo()
```